SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning
NAME: T.Ganesh
REGISTER NUMBER:192425246

COURSE OUTCOME COVERED

CO1: *Analyse reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

   (10 Marks)
2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

   (10 Marks)
3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

   (10 Marks)
4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

   (10 Marks)
5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

   (10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

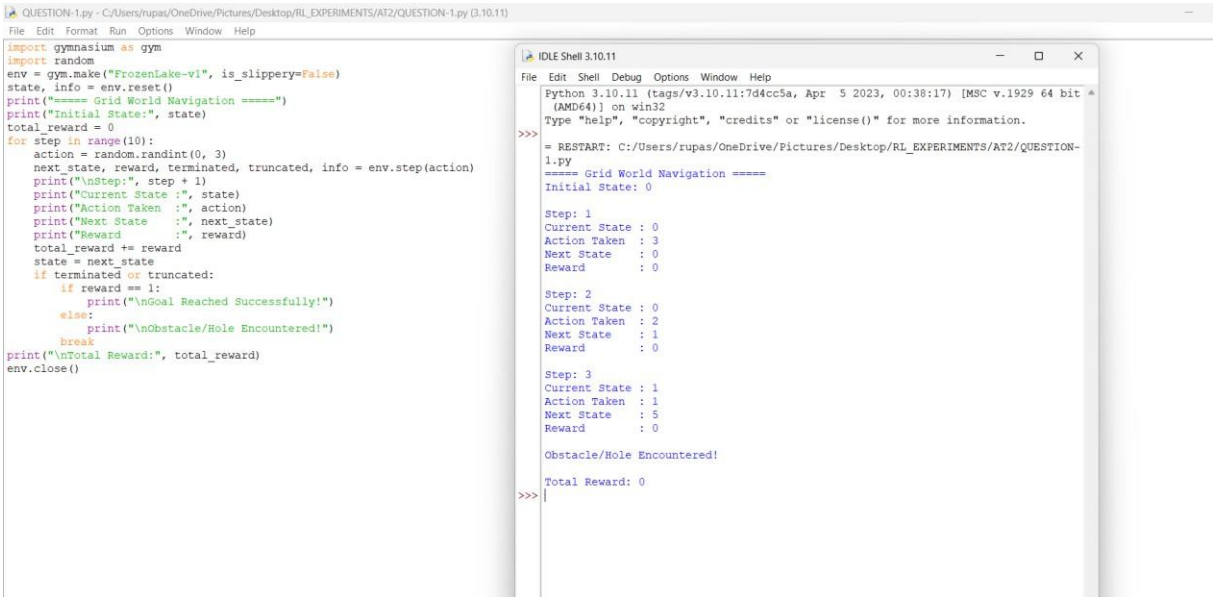
(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

EXPERIMENT - 1

Aim-To develop a Python-based Reinforcement Learning model using the OpenAI Gym environment that enables an agent to navigate a Grid World, reach the goal efficiently, and avoid obstacles by learning an optimal policy.

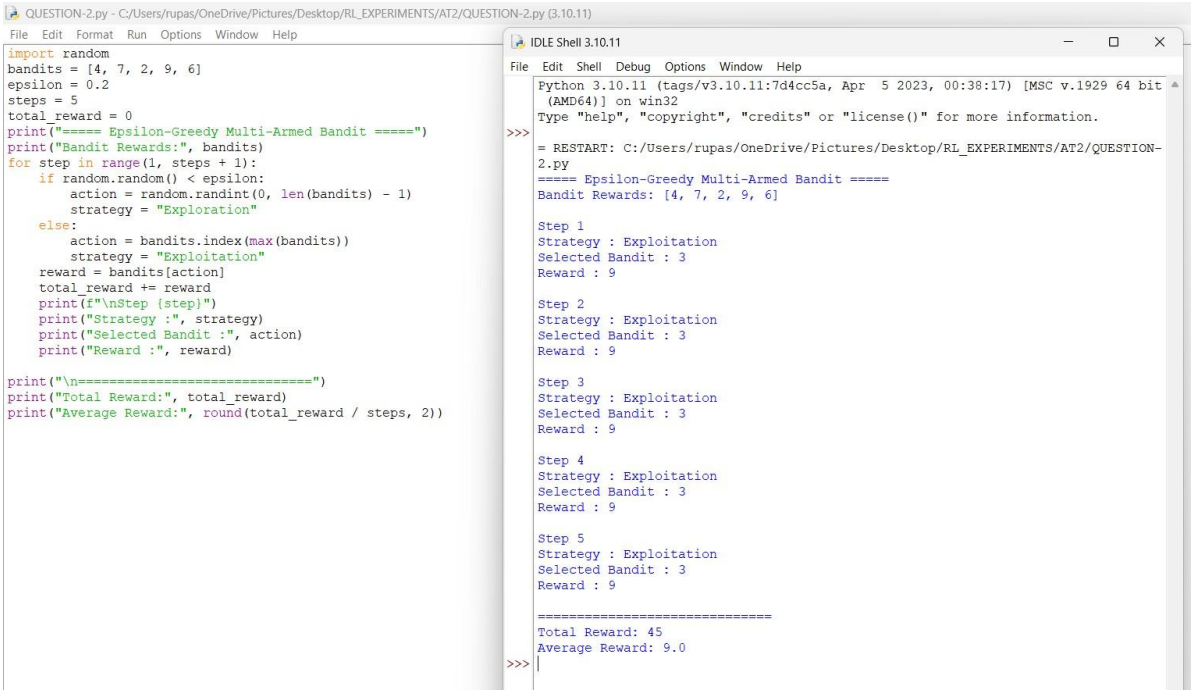


```
QUESTION-1.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-1.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
import gymnasium as gym
import random
env = gym.make("FrozenLake-v1", is_slippery=False)
state, info = env.reset()
print("===== Grid World Navigation =====")
print("Initial State:", state)
total_reward = 0
for step in range(10):
    action = random.randint(0, 3)
    next_state, reward, terminated, truncated, info = env.step(action)
    print("\nStep:", step + 1)
    print("Current State :", state)
    print("Action Taken  :", action)
    print("Next State    :", next_state)
    print("Reward        :", reward)
    total_reward += reward
    state = next_state
    if terminated or truncated:
        if reward == 1:
            print("\nGoal Reached Successfully!")
        else:
            print("\nObstacle/Hole Encountered!")
        break
print("\nTotal Reward:", total_reward)
env.close()
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-
1.py
===== Grid World Navigation =====
Initial State: 0

Step: 1
Current State : 0
Action Taken  : 3
Next State    : 0
Reward        : 0

Step: 2
Current State : 0
Action Taken  : 2
Next State    : 1
Reward        : 0

Step: 3
Current State : 1
Action Taken  : 1
Next State    : 5
Reward        : 0

Obstacle/Hole Encountered!

Total Reward: 0
>>>
```

EXPERIMENT - 2

Aim- To implement the ε-greedy Multi-Armed Bandit algorithm in Python and compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and cumulative rewards under a limited budget of 500 trials.

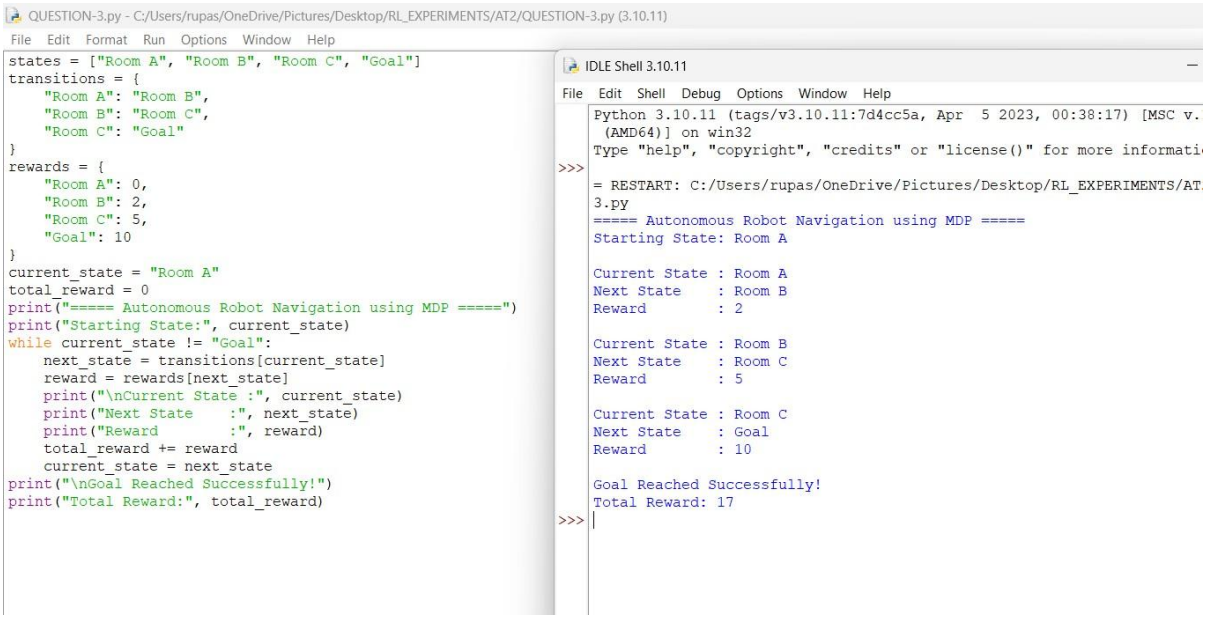


```
QUESTION-2.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-2.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
import random
bandits = [4, 7, 2, 9, 6]
epsilon = 0.2
steps = 5
total_reward = 0
print("===== Epsilon-Greedy Multi-Armed Bandit =====")
print("Bandit Rewards:", bandits)
for step in range(1, steps + 1):
    if random.random() < epsilon:
        action = random.randint(0, len(bandits) - 1)
        strategy = "Exploration"
    else:
        action = bandits.index(max(bandits))
        strategy = "Exploitation"
    reward = bandits[action]
    total_reward += reward
    print(f"\nStep {step}")
    print("Strategy :", strategy)
    print("Selected Bandit :", action)
    print("Reward :", reward)

print("\n==============================")
print("Total Reward:", total_reward)
print("Average Reward:", round(total_reward / steps, 2))
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-
2.py
===== Epsilon-Greedy Multi-Armed Bandit =====
Bandit Rewards: [4, 7, 2, 9, 6]

Step 1
Strategy : Exploitation
Selected Bandit : 3
Reward : 9

Step 2
Strategy : Exploitation
Selected Bandit : 3
Reward : 9

Step 3
Strategy : Exploitation
Selected Bandit : 3
Reward : 9

Step 4
Strategy : Exploitation
Selected Bandit : 3
Reward : 9

Step 5
Strategy : Exploitation
Selected Bandit : 3
Reward : 9

==============================
Total Reward: 45
Average Reward: 9.0
>>>
```

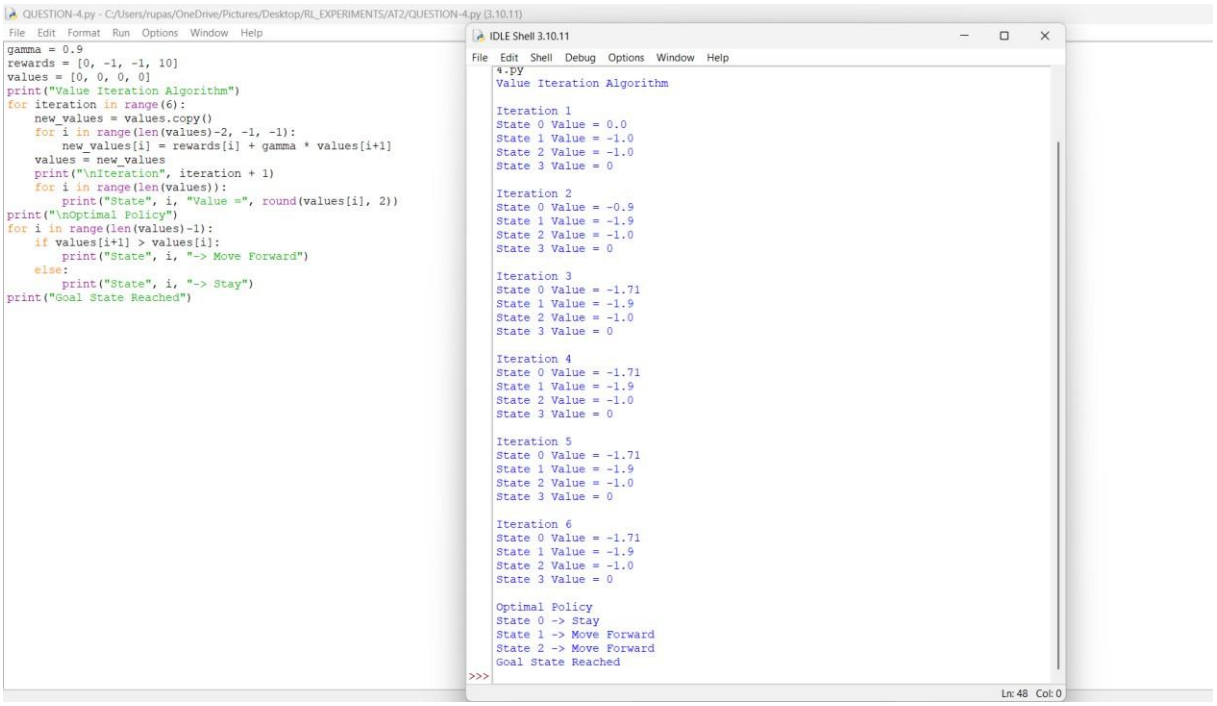
EXPERIMENT - 3

Aim- To design a Markov Decision Process (MDP) model for an autonomous robot that reaches its destination while minimizing energy consumption by defining appropriate states, actions, transition probabilities, rewards, and constraints.



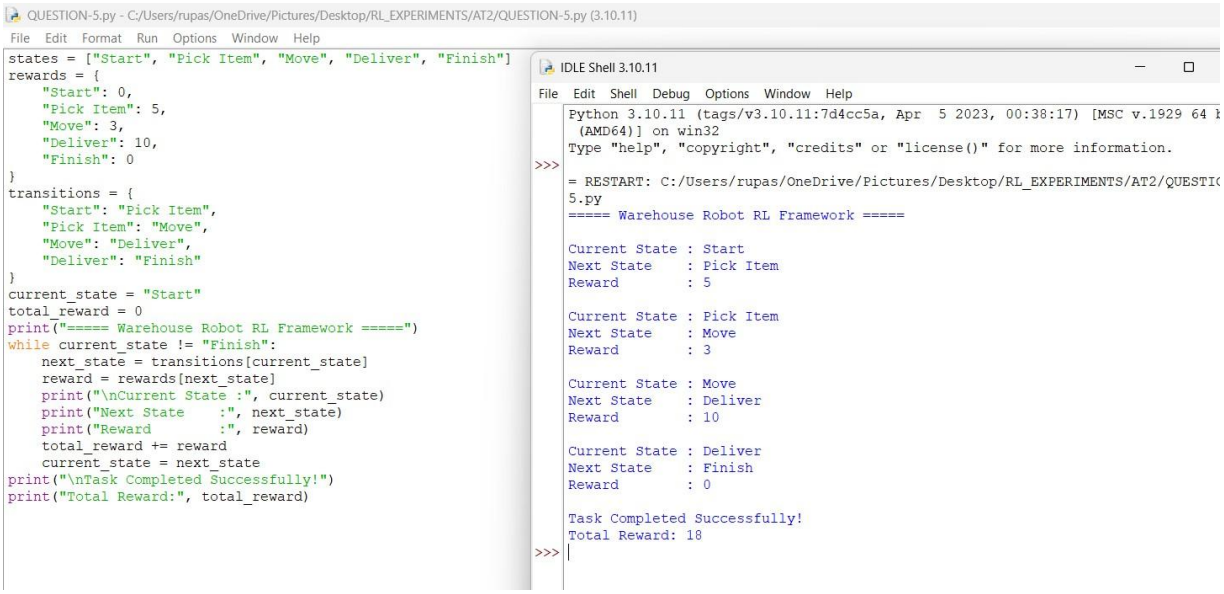
EXPERIMENT - 4

Aim- To implement the Value Iteration algorithm using Python and TensorFlow/Keras for a constrained grid environment and determine the optimal policy using the Bellman equation.



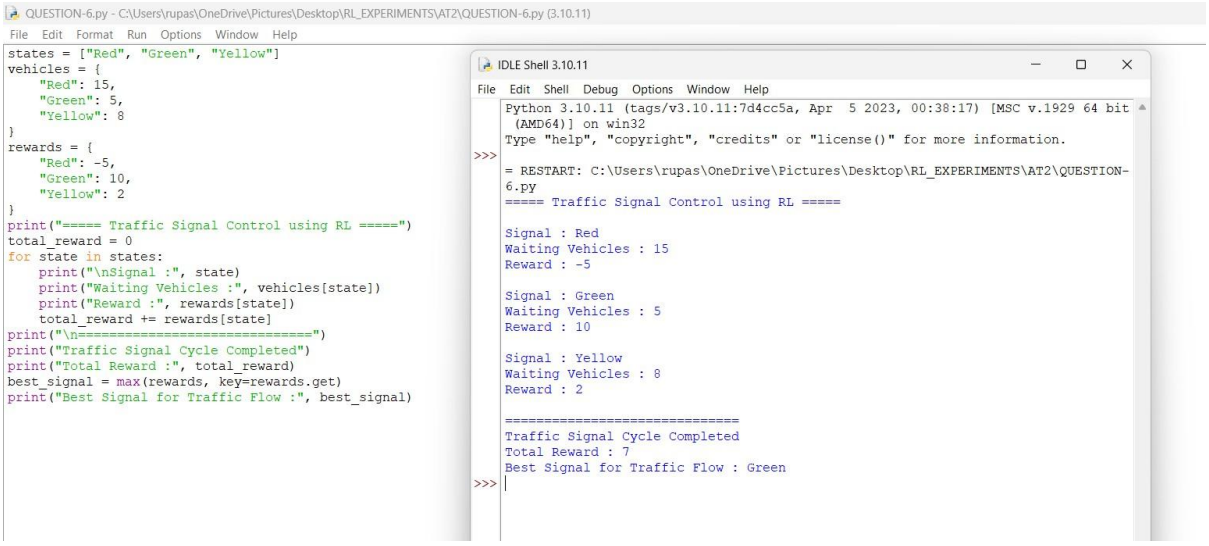
EXPERIMENT - 5

Aim- To design a Reinforcement Learning framework that enables a warehouse robot to maximize the number of deliveries while operating within a 30-minute battery constraint.



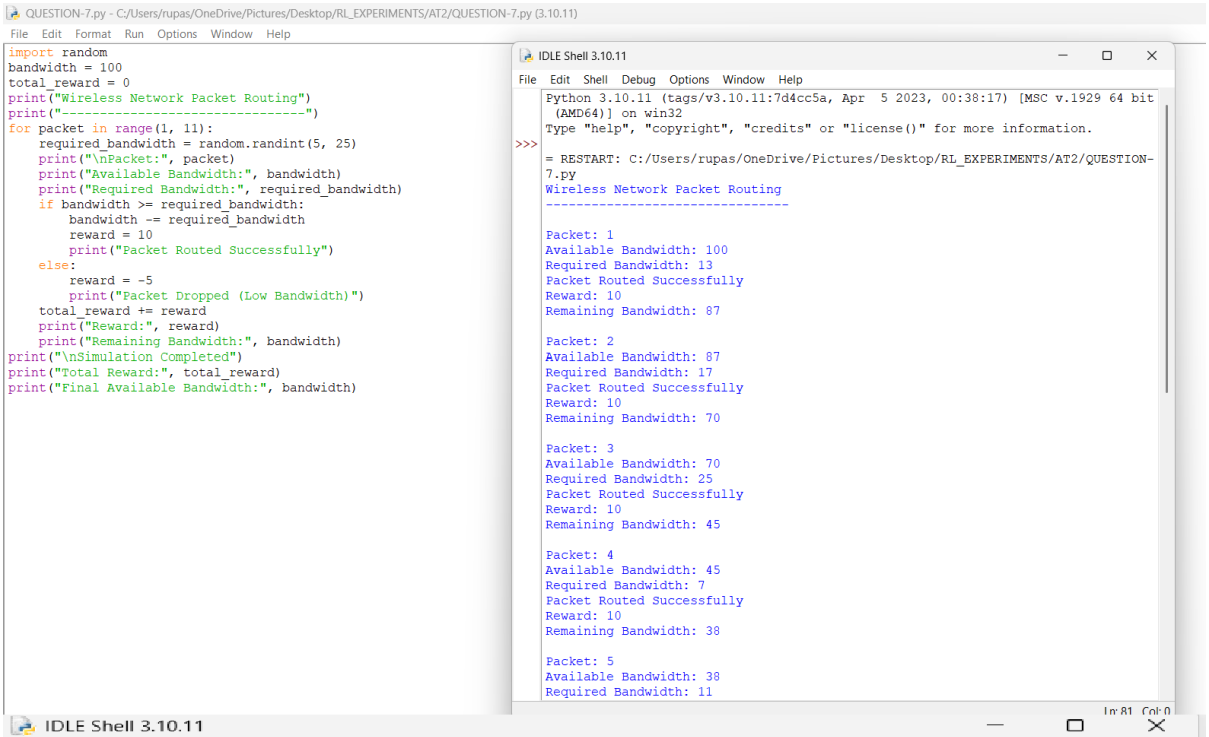
EXPERIMENT - 6

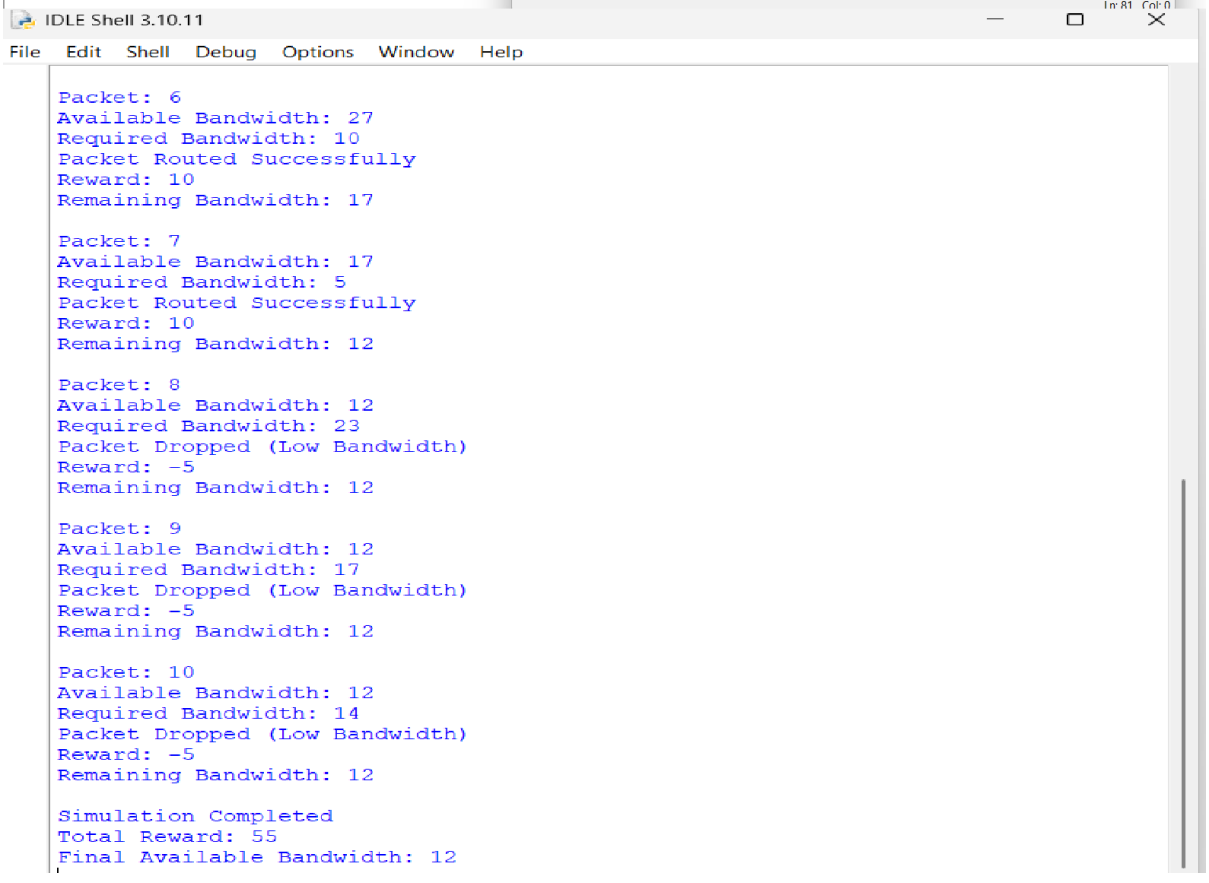
Aim- To implement a Reinforcement Learning agent for intelligent traffic signal control that minimizes traffic congestion while ensuring immediate priority for emergency vehicles.



EXPERIMENT - 7

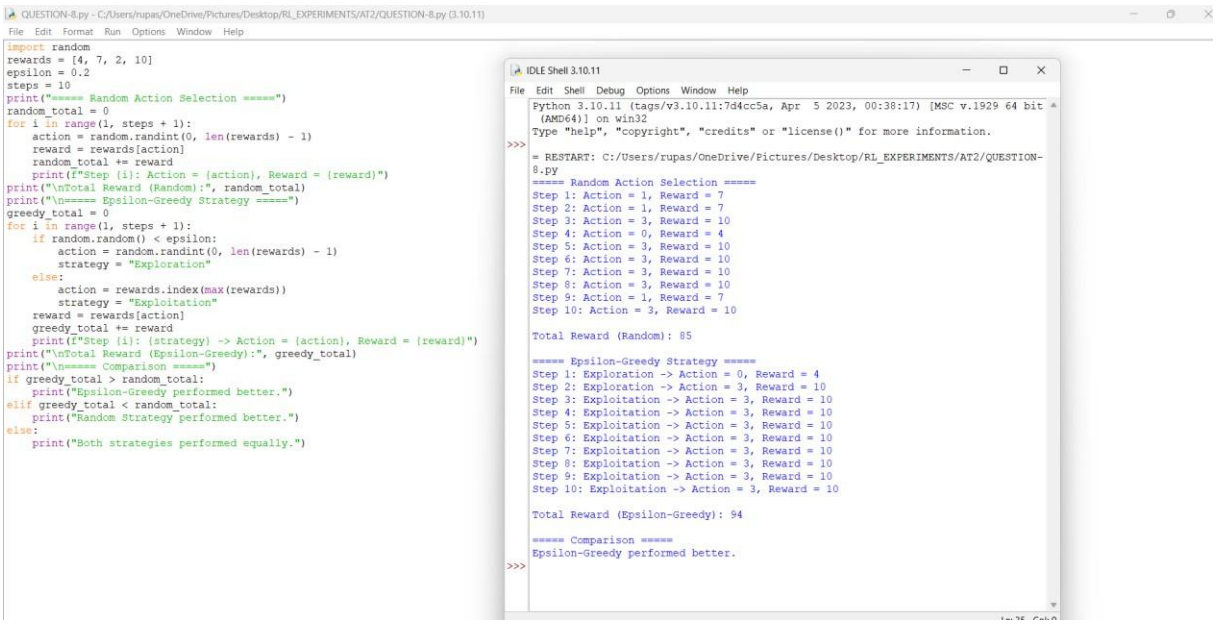
Aim- To develop a Reinforcement Learning-based packet routing system using Python and TensorFlow/Keras that optimizes data transmission while considering limited network bandwidth.





EXPERIMENT - 8

Aim-To analyse and compare the performance of random action selection and ε-greedy action selection in a Multi-Armed Bandit problem by evaluating cumulative rewards under a fixed time constraint.

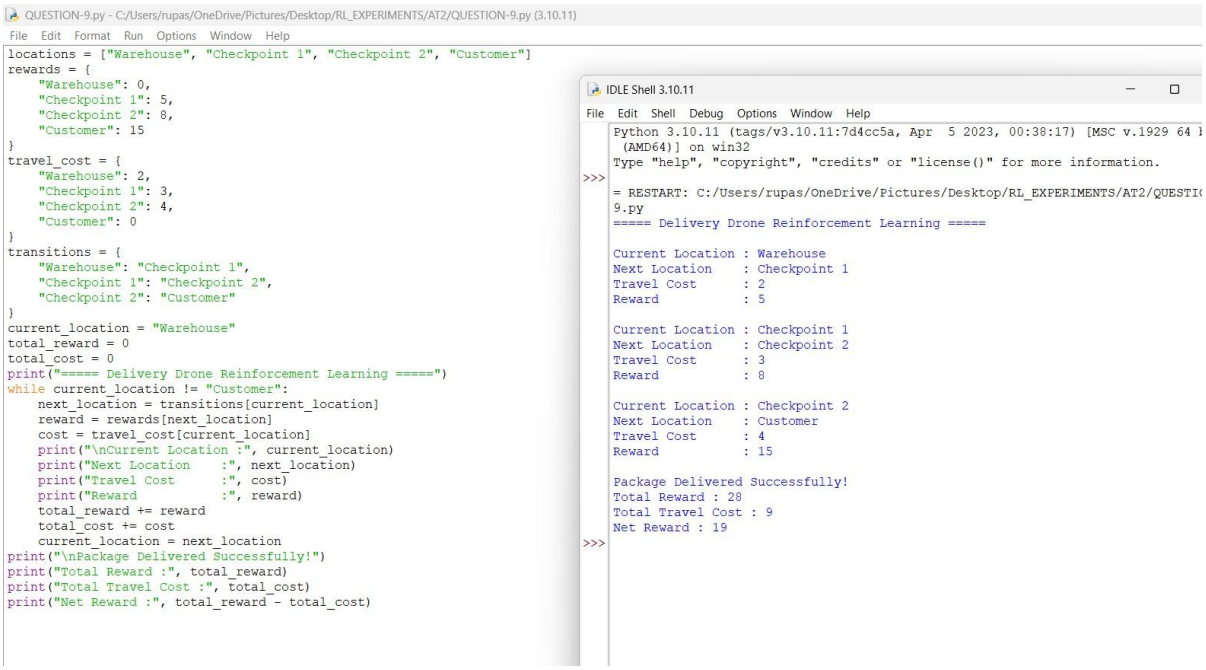


```
QUESTION-8.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-8.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
import random
rewards = [4, 7, 2, 10]
epsilon = 0.2
steps = 10
print("===== Random Action Selection =====")
random_total = 0
for i in range(1, steps + 1):
    action = random.randint(0, len(rewards) - 1)
    reward = rewards[action]
    random_total += reward
    print(f"Step {i}: Action = {action}, Reward = {reward}")
print("\nTotal Reward (Random):", random_total)
print("\n===== Epsilon-Greedy Strategy =====")
greedy_total = 0
for i in range(1, steps + 1):
    if random.random() < epsilon:
        action = random.randint(0, len(rewards) - 1)
        strategy = "Exploration"
    else:
        action = rewards.index(max(rewards))
        strategy = "Exploitation"
    reward = rewards[action]
    greedy_total += reward
    print(f"Step {i}: {strategy} -> Action = {action}, Reward = {reward}")
print("\nTotal Reward (Epsilon-Greedy):", greedy_total)
print("\n===== Comparison =====")
if greedy_total > random_total:
    print("Epsilon-Greedy performed better.")
elif greedy_total < random_total:
    print("Random Strategy performed better.")
else:
    print("Both strategies performed equally.")
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-
8.py
===== Random Action Selection =====
Step 1: Action = 1, Reward = 7
Step 2: Action = 1, Reward = 7
Step 3: Action = 3, Reward = 10
Step 4: Action = 0, Reward = 4
Step 5: Action = 3, Reward = 10
Step 6: Action = 3, Reward = 10
Step 7: Action = 3, Reward = 10
Step 8: Action = 3, Reward = 10
Step 9: Action = 1, Reward = 7
Step 10: Action = 3, Reward = 10

Total Reward (Random): 85

===== Epsilon-Greedy Strategy =====
Step 1: Exploration -> Action = 0, Reward = 4
Step 2: Exploration -> Action = 3, Reward = 10
Step 3: Exploitation -> Action = 3, Reward = 10
Step 4: Exploitation -> Action = 3, Reward = 10
Step 5: Exploitation -> Action = 3, Reward = 10
Step 6: Exploitation -> Action = 3, Reward = 10
Step 7: Exploitation -> Action = 3, Reward = 10
Step 8: Exploitation -> Action = 3, Reward = 10
Step 9: Exploitation -> Action = 3, Reward = 10
Step 10: Exploitation -> Action = 3, Reward = 10

Total Reward (Epsilon-Greedy): 94

===== Comparison =====
Epsilon-Greedy performed better.
>>>
```

EXPERIMENT - 9

Aim-To design a Reinforcement Learning framework for a delivery drone that maximizes successful deliveries while avoiding no-fly zones and operating within battery limitations.

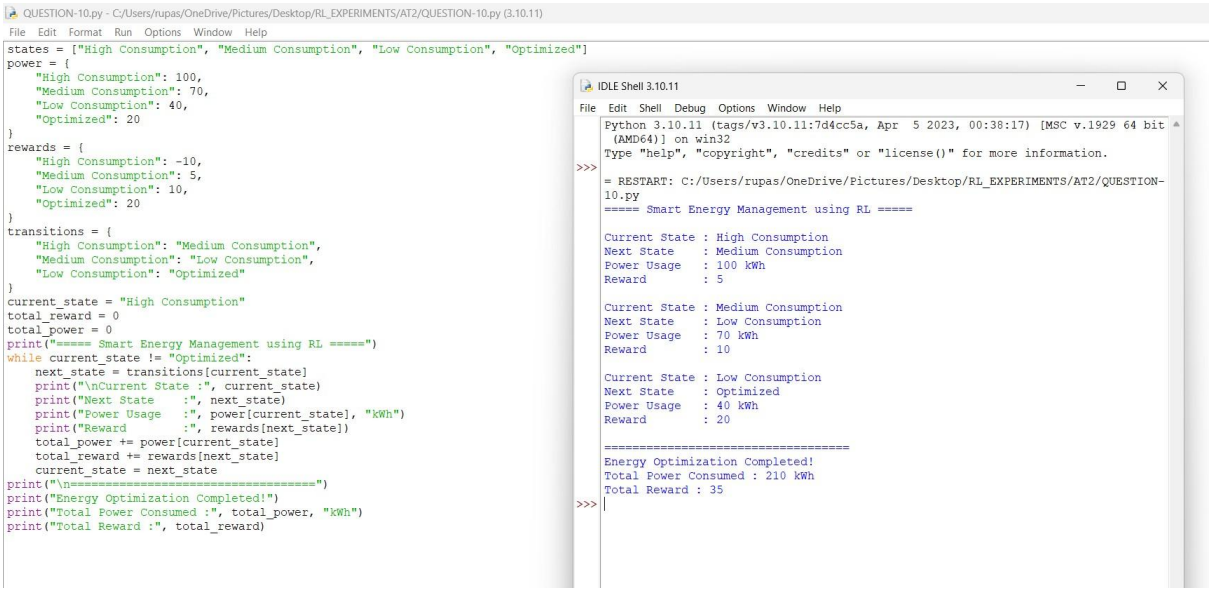


```
QUESTION-9.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-9.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
locations = ["Warehouse", "Checkpoint 1", "Checkpoint 2", "Customer"]
rewards = {
    "Warehouse": 0,
    "Checkpoint 1": 5,
    "Checkpoint 2": 8,
    "Customer": 15
}
travel_cost = {
    "Warehouse": 2,
    "Checkpoint 1": 3,
    "Checkpoint 2": 4,
    "Customer": 0
}
transitions = {
    "Warehouse": "Checkpoint 1",
    "Checkpoint 1": "Checkpoint 2",
    "Checkpoint 2": "Customer"
}
current_location = "Warehouse"
total_reward = 0
total_cost = 0
print("===== Delivery Drone Reinforcement Learning =====")
while current_location != "Customer":
    next_location = transitions[current_location]
    reward = rewards[next_location]
    cost = travel_cost[current_location]
    print("\nCurrent Location :", current_location)
    print("Next Location    :", next_location)
    print("Travel Cost      :", cost)
    print("Reward           :", reward)
    total_reward += reward
    total_cost += cost
    current_location = next_location
print("\nPackage Delivered Successfully!")
print("Total Reward :", total_reward)
print("Total Travel Cost :", total_cost)
print("Net Reward :", total_reward - total_cost)
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 b
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTI
9.py
===== Delivery Drone Reinforcement Learning =====

Current Location : Warehouse
Next Location    : Checkpoint 1
Travel Cost      : 2
Reward           : 5

Current Location : Checkpoint 1
Next Location    : Checkpoint 2
Travel Cost      : 3
Reward           : 8

Current Location : Checkpoint 2
Next Location    : Customer
Travel Cost      : 4
Reward           : 15

Package Delivered Successfully!
Total Reward : 28
Total Travel Cost : 9
Net Reward : 19
>>>
```

EXPERIMENT - 10

Aim-To implement a Reinforcement Learning solution for a smart energy management system that minimizes electricity consumption while maintaining usage below a specified threshold through reward shaping and policy optimization.



```
QUESTION-10.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-10.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
states = ["High Consumption", "Medium Consumption", "Low Consumption", "Optimized"]
power = {
    "High Consumption": 100,
    "Medium Consumption": 70,
    "Low Consumption": 40,
    "Optimized": 20
}
rewards = {
    "High Consumption": -10,
    "Medium Consumption": 5,
    "Low Consumption": 10,
    "Optimized": 20
}
transitions = {
    "High Consumption": "Medium Consumption",
    "Medium Consumption": "Low Consumption",
    "Low Consumption": "Optimized"
}
current_state = "High Consumption"
total_reward = 0
total_power = 0
print("===== Smart Energy Management using RL =====")
while current_state != "Optimized":
    next_state = transitions[current_state]
    print("\nCurrent State :", current_state)
    print("Next State    :", next_state)
    print("Power Usage   :", power[current_state], "kWh")
    print("Reward        :", rewards[next_state])
    total_power += power[current_state]
    total_reward += rewards[next_state]
    current_state = next_state
print("\n==================================")
print("Energy Optimization Completed!")
print("Total Power Consumed :", total_power, "kWh")
print("Total Reward :", total_reward)
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/AT2/QUESTION-
10.py
===== Smart Energy Management using RL =====

Current State : High Consumption
Next State    : Medium Consumption
Power Usage   : 100 kWh
Reward        : 5

Current State : Medium Consumption
Next State    : Low Consumption
Power Usage   : 70 kWh
Reward        : 10

Current State : Low Consumption
Next State    : Optimized
Power Usage   : 40 kWh
Reward        : 20

==================================
Energy Optimization Completed!
Total Power Consumed : 210 kWh
Total Reward : 35
>>>
```

Code of Conduct:

I T.Ganesh(192425246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T.Ganesh

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided